```
        }
    }
}
```

❶ Background-Service is registered as a singleton.

❷ The injected IServiceProvider can be used to retrieve singleton services or to create scopes.

❸ Creates a new scope using the root IServiceProvider

❹ The scope exposes an IServiceProvider that can be used to retrieve scoped components.

❺ Retrieves the scoped services from the container

❻ Fetches the latest rates, and saves using EF Core

❼ Disposes of the scope with the using statement

❽ Waits for the next iteration. A new scope is created on the next iteration.

Creating scopes like this is a general solution whenever you need to access scoped services and you're not running in the context of a request. For example, if you need to access scoped or transient services in Program.cs, you can create a new scope by calling `WebApplication.Services.CreateScope()`. You can then retrieve the services you need, do your work, and dispose the scope to clean up the services.

Another prime example is when you're injecting services into an `OptionsBuilder` instance, as you saw in chapter 31. You can take exactly the same approach—create a new scope—as shown in my blog post titled "The dangers and gotchas of using scoped services in `OptionsBuilder`": http://mng.bz/4D6j.

TIP Using service location in this way always feels a bit convoluted. I typically try to extract the body of the task to a separate class and use service location to retrieve that class only. You can see an example of this approach in the "Consuming a scoped service in a background task" section of Microsoft's "Background tasks with hosted services in ASP.NET Core" documentation: http://mng.bz/4ZER.